

AI ASSISTED CODING

2303A52346

K.SUJITH

ASS 10.3

AI-Assisted Bug Detection

Scenario: A junior developer wrote the following Python function to calculate factorials:

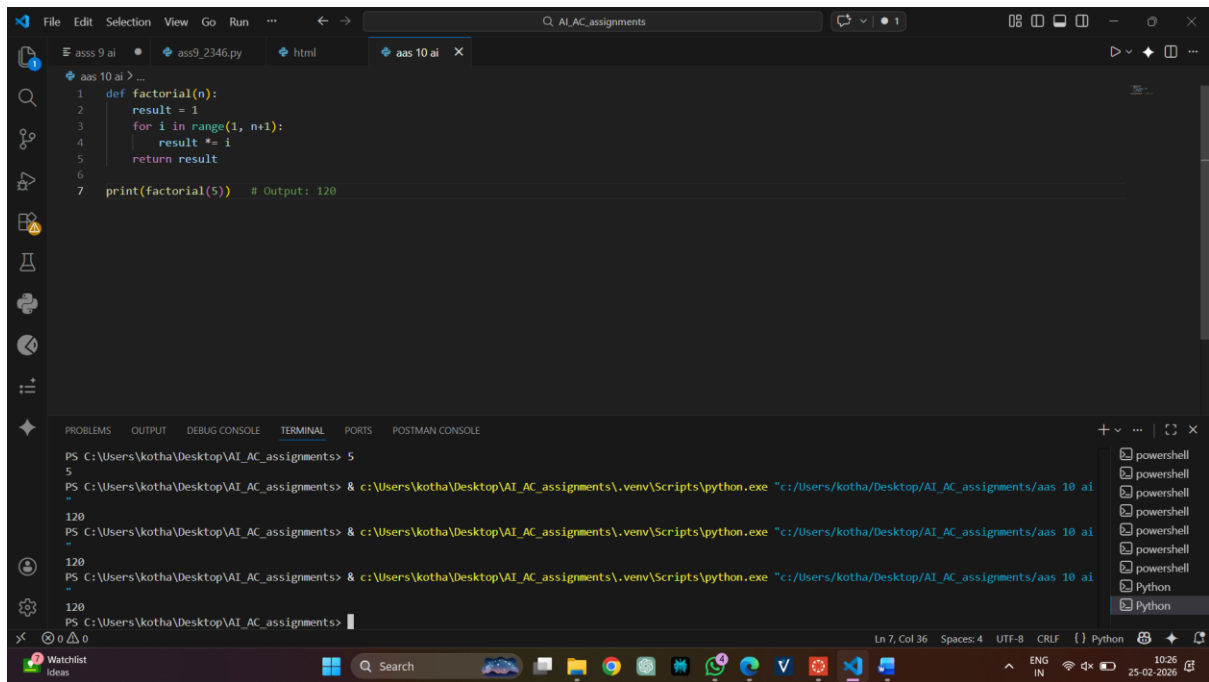
```
def factorial(n):  
    result = 1  
    for i in range(1, n):  
        result = result * i  
    return result
```

Instructions:

1. Run the code and test it with factorial(5).
2. Use an AI assistant to:
 - o Identify the logical bug in the code.
 - o Explain why the bug occurs (e.g., off-by-one error).
 - o Provide a corrected version.
3. Compare the AI's corrected code with your own manual fix.
4. Write a brief comparison: Did AI miss any edge cases (e.g., negative numbers, zero)?

Expected Output:

Corrected function should return 120 for factorial(5).



Improving Readability &

Documentation

Scenario: The following code works but is poorly written:

```
.  
  
def calc(a, b, c):  
    if c == "add":  
        return a + b  
    elif c == "sub":  
        return a - b  
    elif c == "mul":  
        return a * b  
    elif c == "div":
```

Instructions:

5. Use AI to:

- o Critique the function's readability, parameter naming, and lack of documentation.
- o Rewrite the function with:
 1. Descriptive function and parameter names.
 2. A complete docstring (description, parameters,

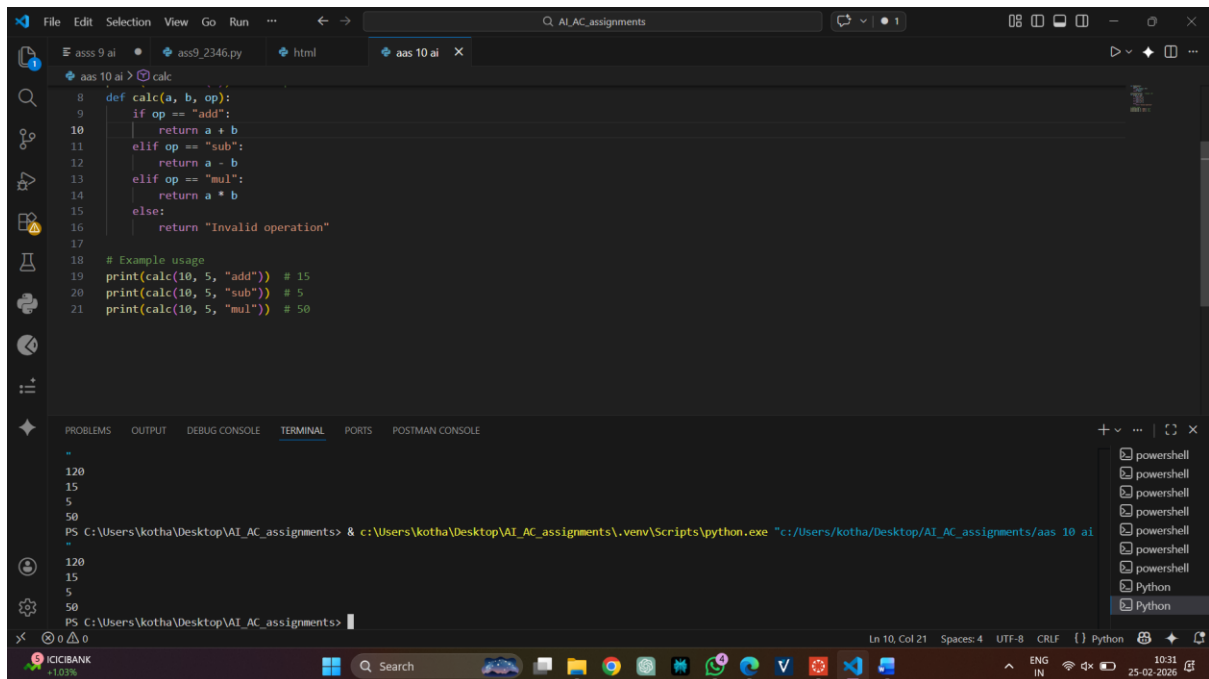
return value, examples).

3. Exception handling for division by zero.

4. Consideration of input validation.

6. Compare the original and AI-improved versions.

7. Test both with valid and invalid inputs (e.g., division by zero, non-string operation).



```
File Edit Selection View Go Run ...  
AI AC assignments  
ass9 ai ass9_2346.py html aas 10 ai x  
aas 10 ai > calc  
8 def calc(a, b, op):  
9     if op == "add":  
10         return a + b  
11     elif op == "sub":  
12         return a - b  
13     elif op == "mul":  
14         return a * b  
15     else:  
16         return "Invalid operation"  
17  
18 # Example usage  
19 print(calc(10, 5, "add")) # 15  
20 print(calc(10, 5, "sub")) # 5  
21 print(calc(10, 5, "mul")) # 50  
22  
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE  
+  
120  
15  
5  
50  
PS C:\Users\kotha\Desktop\AI_AC_assignments> & c:\Users\kotha\Desktop\AI_AC_assignments\.venv\Scripts\python.exe "c:\Users\kotha\Desktop\AI_AC_assignments\aas 10 ai"  
+  
120  
15  
5  
50  
PS C:\Users\kotha\Desktop\AI_AC_assignments>  
Ln 10, Col 21 Spaces: 4 UTF-8 CRLF Python
```

I as a Code Reviewer in Real Projects

Scenario:

In a GitHub project, a teammate submits:

```
def processData(d):
```

```
    return [x * 2 for x in d if x % 2 == 0]
```

Instructions:

1. Manually review the function for:

- o Readability and naming.
- o Reusability and modularity.
- o Edge cases (non-list input, empty list, non-integer elements).

2. Use AI to generate a code review covering:

- a. Better naming and function purpose clarity.

- b. Input validation and type hints.
 - c. Suggestions for generalization (e.g., configurable multiplier).
3. Refactor the function based on AI feedback.
 4. Write a short reflection on whether AI should be a standalone reviewer or an assistant.

Expected Output:

An improved function with type hints, validation, and clearer intent,

```

22 def is_prime(n):
23     if n <= 1:
24         return False
25     for i in range(2, int(n**0.5) + 1): # check only till √n
26         if n % i == 0:
27             return False
28     return True
29
30 # Example
31 num = 29
32 print("Prime" if is_prime(num) else "Not Prime")
33

```

```

120
15
5
50
Prime
PS C:\Users\kotha\Desktop\AI_AC_assignments> & c:\Users\kotha\Desktop\AI_AC_assignments\.venv\Scripts\python.exe "c:/Users/kotha/Desktop/AI_AC_assignments/aas 10 ai"
File "c:\Users\kotha\Desktop\AI_AC_assignments\aa 10 ai", line 36
    if not isinstance(numbers, list):
    ^
IndentationError: expected an indented block after function definition on line 34
PS C:\Users\kotha\Desktop\AI_AC_assignments>

```

AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but

it runs slowly on large datasets:

```
def sum_of_squares(numbers):
```

```
    total = 0
```

```
    for num in numbers:
```

```
        total += num ** 2
```

```
    return total
```

